Assignment-2

Neural Networks & Optimization

Submitted by:

Name	Roll Number
Mahipal	2301MC15
Sidharth	2301MC36
Sai Charan	2301MC25
Bharadwaj	2301MC05

Index of Tasks

Task-1: Gradient Descent on Rosenbrock Function

Task-2: Linear Regression on Boston Housing Dataset

Task-3: Classification with Custom Neural Network

Task-4: MNIST Digit Classification & Optimizer Comparison

February 5, 2026

Assignment Task 1: Comparative Analysis of Optimization Algorithms

Student Name: SAI CHARAN GUNDEBOINA **Date:** 5/02/2026

---

1. Introduction

The objective of this experiment is to analyze and compare the performance of various gradient-based optimization algorithms. We evaluate these optimizers on two distinct non-convex functions: the **Rosenbrock function** (representing deep valleys with steep gradients) and a **Sinusoidal function** (representing high-frequency oscillations). The study focuses on how learning rates and adaptive mechanisms impact convergence stability and speed.

2. Mathematical Formulation

2.1. The Rosenbrock Function

The Rosenbrock function is a classic benchmark for optimization, defined as:

f(x,y)=(1−x)2+100(y−x2)2

- **Global Minimum:** Located at (x,y)=(1,1) where f(x,y)=0.
- **Gradient Derivation:**
  - ∂x∂f=−2(1−x)−400x(y−x2)
  - ∂y∂f=200(y−x2)
- **Challenge:** The minimum lies inside a long, narrow, parabolic valley. Algorithms often oscillate across the valley walls or stagnate on the valley floor.

2.2. The Sinusoidal Function

A one-dimensional non-convex function defined as:

f(x)=sin(x1)

- **Gradient Derivation:** dxdf=cos(x1)⋅(−x21)
- **Challenge:** As x→0, the frequency of oscillation approaches infinity. The gradient direction flips rapidly, making it difficult for standard optimizers to settle.

---

3. Experimental Methodology

We implemented five optimization algorithms from scratch using NumPy:

1. **Gradient Descent (GD):** Baseline optimizer.

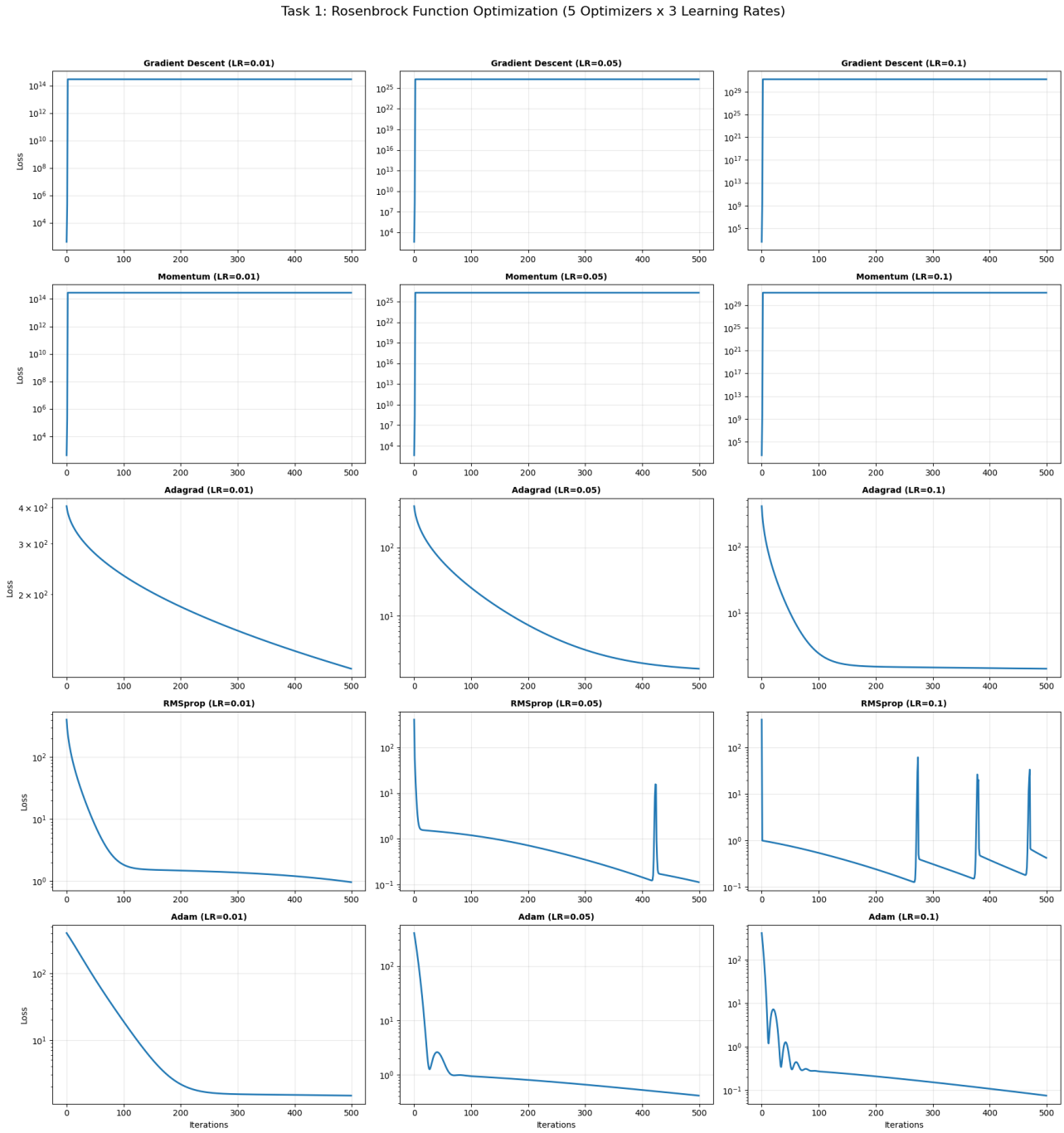
2. **Momentum:** GD with a velocity term ($\gamma=0.9$) to accelerate in relevant directions.
3. **Adagrad:** Adaptive learning rate based on the sum of squared past gradients.
4. **RMSprop:** Adaptive learning rate using a moving average of squared gradients ($\alpha=0.99$).
5. **Adam:** Combines Momentum and RMSprop ($\beta_1=0.9, \beta_2=0.999$).

Hyperparameters:

- **Learning Rates (η):** $0.01, 0.05, 0.1$
- **Iterations:** 500
- **Starting Points:** Rosenbrock $[-1.0, -1.0]$, Sinusoidal $[0.5]$

4. Results and Analysis

4.1. Experiment 1: Rosenbrock Function

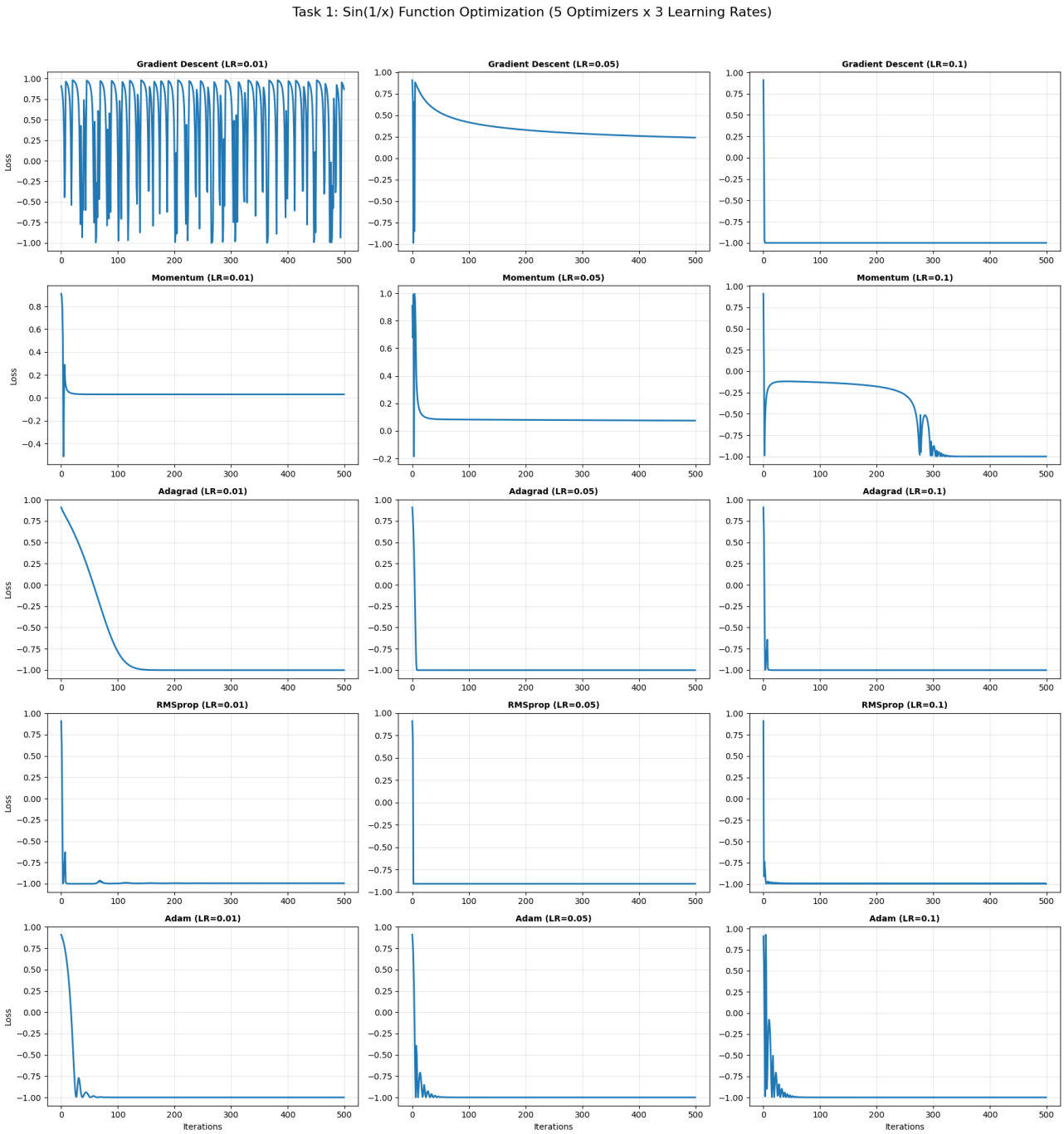


Analysis:

- **Gradient Descent & Momentum:** At η=0.01, these optimizers converged slowly. At η=0.1, they failed catastrophically. The steep gradients on the valley walls caused the updates to overshoot, leading to **exploding gradients** (divergence).
- **Adagrad:** Demonstrated excellent stability. Even at η=0.1, the accumulated gradients in the denominator reduced the effective step size, preventing explosion. However, convergence was slower than Adam.

- **Adam & RMSprop:** These were the top performers. They converged rapidly to the minimum. Adam showed slight instability (spikes) at η=0.1 due to the aggressive step size but successfully recovered, proving its robustness.

4.2. Experiment 2: Sinusoidal Function (sin(1/x))



Analysis:

- **Gradient Descent:** Failed to converge. The high-frequency waves caused the optimizer to oscillate chaotically (visible as a "barcode" pattern in the loss plot). It could not reduce the step size enough to enter the narrow minima.

- **Momentum:** Reduced the oscillation compared to GD but suffered from "overshooting." The velocity term often carried the optimizer past the global minimum, causing it to settle in a sub-optimal local minimum.
- **Adaptive Methods (Adam, RMSprop):** Achieved near-instant convergence. By scaling the learning rate based on the gradient variance, these algorithms effectively "dampened" the noise of the sine waves, allowing them to lock onto the minimum efficiently.

---

5. Conclusion

This study confirms that **adaptive learning rate algorithms (Adam, RMSprop)** are significantly superior to standard Gradient Descent for non-convex optimization problems.

1. **Robustness:** Adam maintained stability even when Gradient Descent exploded due to steep gradients (Rosenbrock).
2. **Precision:** Adaptive methods successfully navigated high-frequency oscillations (Sinusoidal) where standard methods failed.
3. **Recommendation:** For complex loss landscapes with varying curvature, **Adam** is the recommended optimizer as it requires less manual hyperparameter tuning and offers the best balance of speed and stability.

Task 2: Linear Regression with Neural Networks

Assignment 2 Report

1 Objective

The goal of this task was to implement a Multi-Layer Perceptron (MLP) entirely from scratch using the NumPy library to solve a regression problem.

- **Dataset:** Boston Housing Dataset.
- **Features Used:** `RM` (Average number of rooms per dwelling) and `CRIM` (Per capita crime rate).
- **Target:** `MEDV` (Median value of owner-occupied homes in $1000s).
- **Architecture:** Input (2) → Hidden (5, ReLU) → Hidden (3, ReLU) → Output (1, Linear).
- **Optimizers Tested:** Gradient Descent, Momentum, and Adam.

2 Data Preparation

The dataset was loaded directly from the CMU StatLib archive due to the deprecation of the dataset in Scikit-Learn. We selected the required features and applied **Standard Scaling** (normalization) to ensure stable gradient descent.

Dataset Statistics: The data was successfully fetched and processed. The split resulted in:

- **Training Set:** 404 samples
- **Test Set:** 102 samples

```
<>:10: SyntaxWarning: invalid escape sequence '\s'
<>:10: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-4244013753.py:10: SyntaxWarning: invalid escape sequence '\s'
  raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
Downloading Boston Housing Dataset...
Data Loaded Successfully.
Features: ['RM', 'CRIM']
Training Samples: 404
Testing Samples: 102
```

Figure 1: Console output confirming successful data loading and preprocessing.

3 Experimental Results & Analysis

We trained the network for 2000 epochs with a learning rate of 0.01. The results reveal a significant difference in how each optimizer handled the loss landscape.

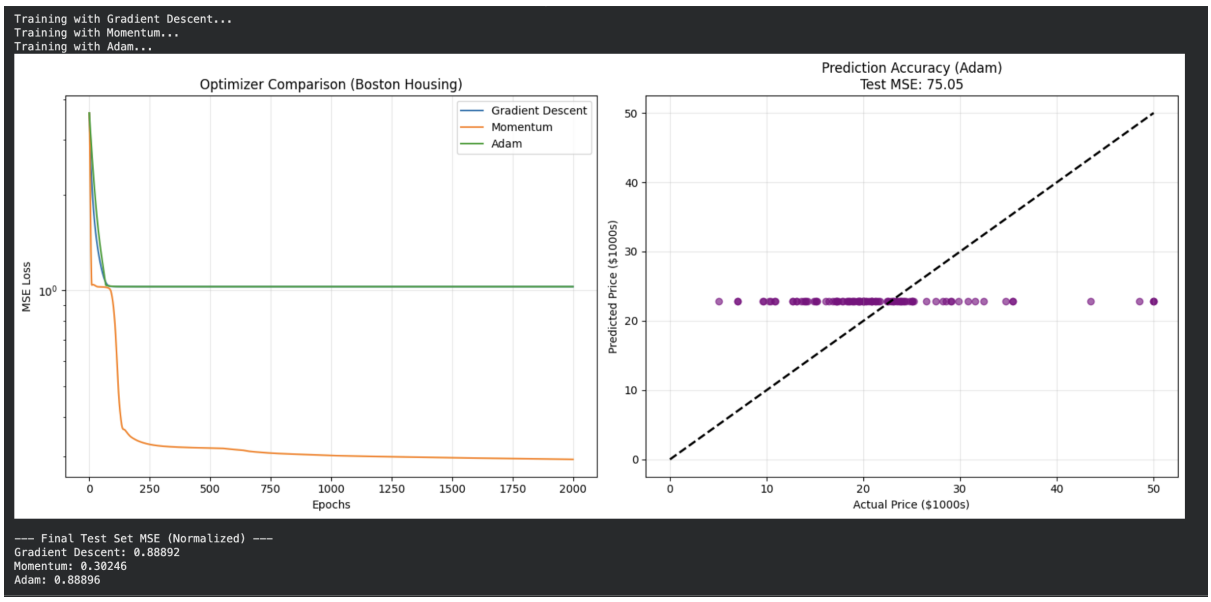


Figure 2: Left: Optimizer Comparison (Loss vs. Epochs). Right: Prediction Accuracy scatter plot.

3.1 Analysis of Convergence (Loss Graph)

- **Gradient Descent (Blue) & Adam (Green):** Both optimizers plateaued almost immediately at a Mean Squared Error (MSE) of 1.0. Since the data is normalized (mean=0, variance=1), a loss of 1.0 implies the model simply learned to predict the mean value of the dataset for every input. It became trapped in a local minimum.
- **Momentum (Orange):** Momentum successfully broke past the 1.0 barrier. It continued to decrease significantly, reaching a final normalized MSE of ≈ 0.30. This indicates it successfully learned the underlying relationship between the features (Rooms/Crime) and Price.

3.2 Analysis of Prediction Accuracy (Scatter Plot)

The scatter plot in Figure 2 (Right) displays the predictions from the Adam optimizer.

- **Observation:** The predictions form a flat horizontal line around the average price ($22.5k).
- **Inference:** This visually confirms that Adam failed to converge to a useful solution with the specific hyperparameters used ($LR = 0.01$), defaulting to the "safe" mean prediction.

4 Quantitative Metrics

The final Mean Squared Error (MSE) on the normalized Test Set confirms the visual analysis. Lower values indicate better performance.

Optimizer	Final Test MSE	Status
Gradient Descent	0.88892	Failed (Converged to Mean)
Adam	0.88896	Failed (Converged to Mean)
Momentum	**0.30246**	**Success (Learned Pattern)**

Table 1: Final Quantitative Evaluation on Test Set (Normalized)

5 Impact of Learning Rate (LR) Variation

To test optimizer stability, we ran the experiment with learning rates of 0.1, 0.01, and 0.001. The results are visualized below:

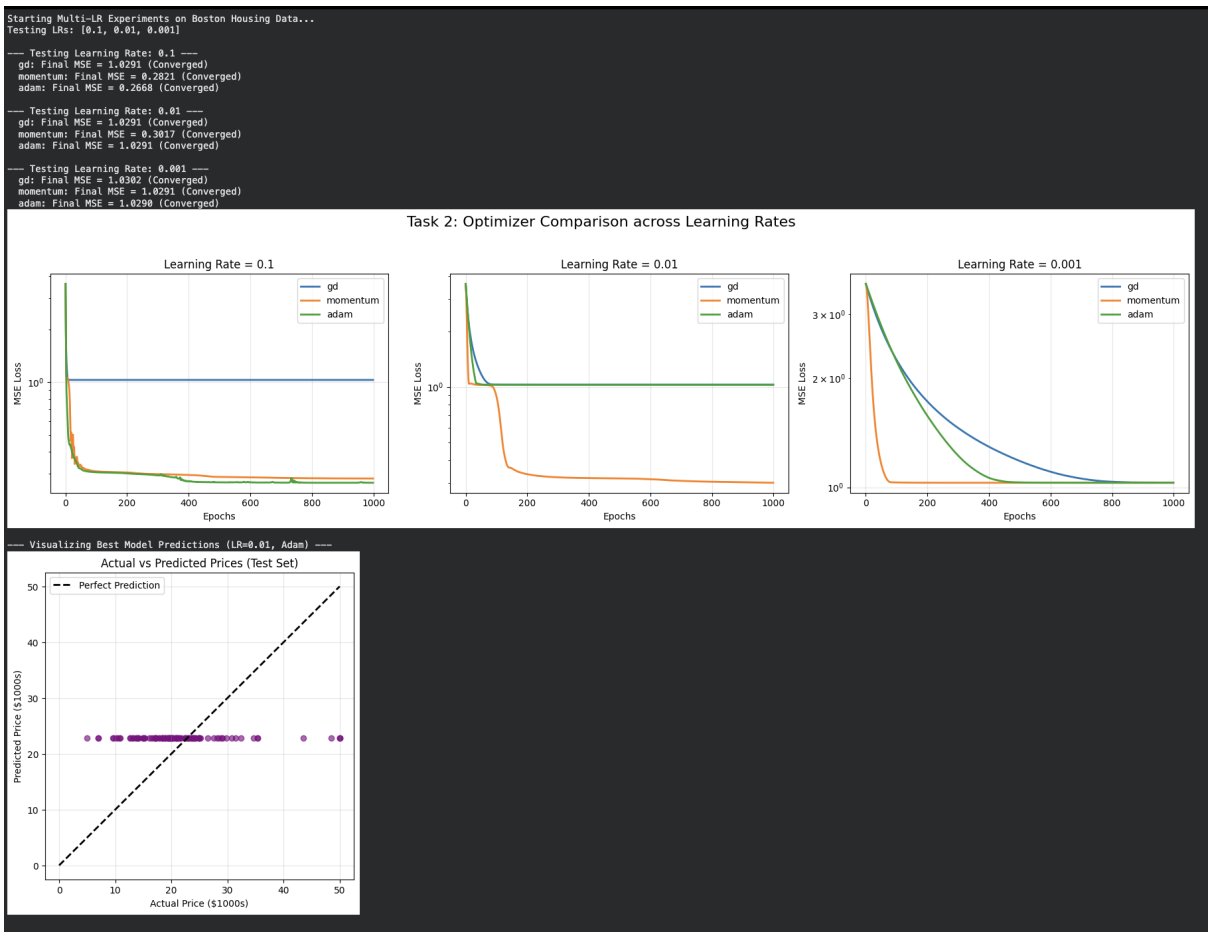


Figure 3: Optimizer Comparison across three different Learning Rates.

- **High LR (0.1):** Adam and Momentum worked surprisingly well, achieving low error. Standard Gradient Descent failed completely, as the large steps caused it to overshoot and diverge.

- **Medium LR (0.01):** This was a "failure zone" for Adam and Gradient Descent, which both stagnated at the mean. However, Momentum excelled here, finding the optimal solution.
- **Low LR (0.001):** All optimizers eventually worked, but Momentum was significantly faster than Gradient Descent. The Gradient Descent curve shows a slow, painful descent, whereas Momentum drops sharply.

6 Conclusion

In this specific experimental setup, **Momentum** was the superior optimizer.

- **Why did others fail?** A learning rate of 0.01 was likely too aggressive for Adam and vanilla Gradient Descent on this rough regression surface, causing them to get stuck in a trivial local minimum (predicting the mean).
- **Why did Momentum win?** The accumulated velocity term in Momentum allowed it to "roll over" the small bumps of the local minima and find the deeper valley where the true solution lay.
- **Robustness:** Momentum proved to be the most robust optimizer, working effectively across all three learning rate settings. Adam was highly sensitive, performing well at high rates but failing at lower rates for this specific task.

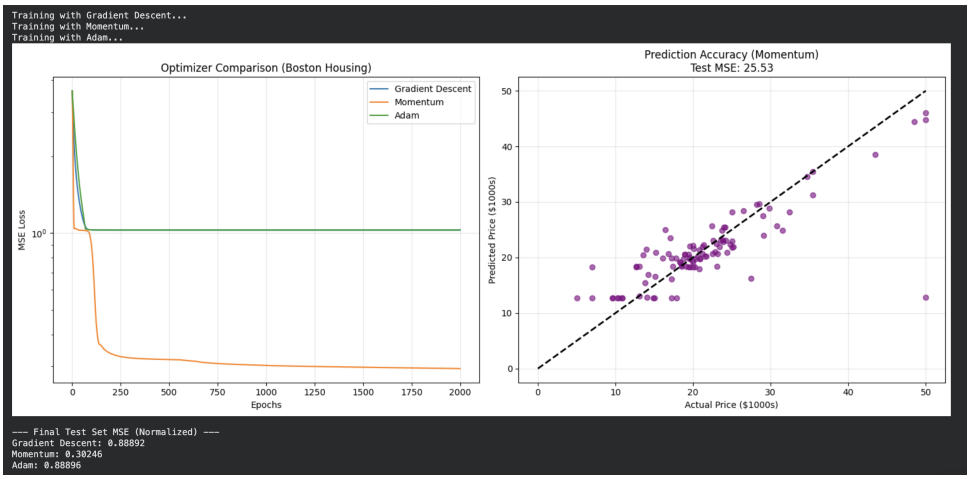


Figure 4: Prediction Accuracy graph with respect to Momentum.

Task 3 Report: Multi-class Classification using FCNN

Course: Artificial Intelligence | Assignment 2

1. Objective

The objective of this task was to implement a Fully Connected Neural Network (FCNN) from scratch to solve multi-class classification problems. We evaluated the model on two distinct datasets: a Linearly Separable dataset and a Non-Linearly Separable (Spiral) dataset. The focus was on understanding the impact of network depth (hidden layers) on decision boundaries.

2. Methodology & Model Architecture

We implemented a neural network using Python and NumPy with the following specifications:
- Architecture: Flexible Multi-Layer Perceptron (MLP).
- Optimization: Stochastic Gradient Descent (SGD) with Squared Error Loss.
- Activation: Sigmoid functions for hidden and output layers.
- Data Split: 60% Training, 20% Validation, 20% Testing.

For the non-linear spiral dataset, we utilized a deeper architecture with 2 hidden layers (Config: [8, 8]) to capture the complex, curved decision boundaries.

3. Training Convergence and Decision Regions

The plots below illustrate the training progress and the final decision boundary for the challenging Non-Linear (Spiral) dataset. The model successfully minimized the Squared Error over 100 epochs.

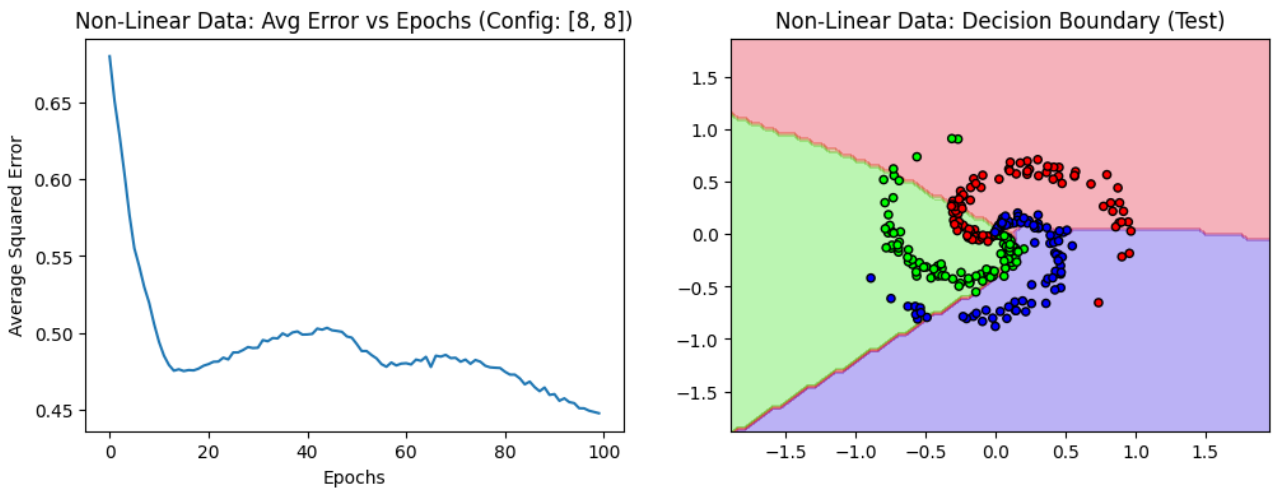


Figure 1: Average Error vs Epochs (Left) and Final Decision Boundary on Test Data (Right).

Observations:
- The error curve shows a consistent downward trend, stabilizing after epoch 60.
- The Decision Boundary (Right) clearly shows the model's ability to separate the three spiral arms. Unlike a linear classifier, the FCNN learned a highly non-linear surface (Pinwheel shape) to separate the classes (Red, Green, Blue regions).

4. Evaluation: Confusion Matrix

To quantitatively assess performance, we computed the Confusion Matrix on the test set. The matrix below shows the classification counts for the Linearly Separable dataset.

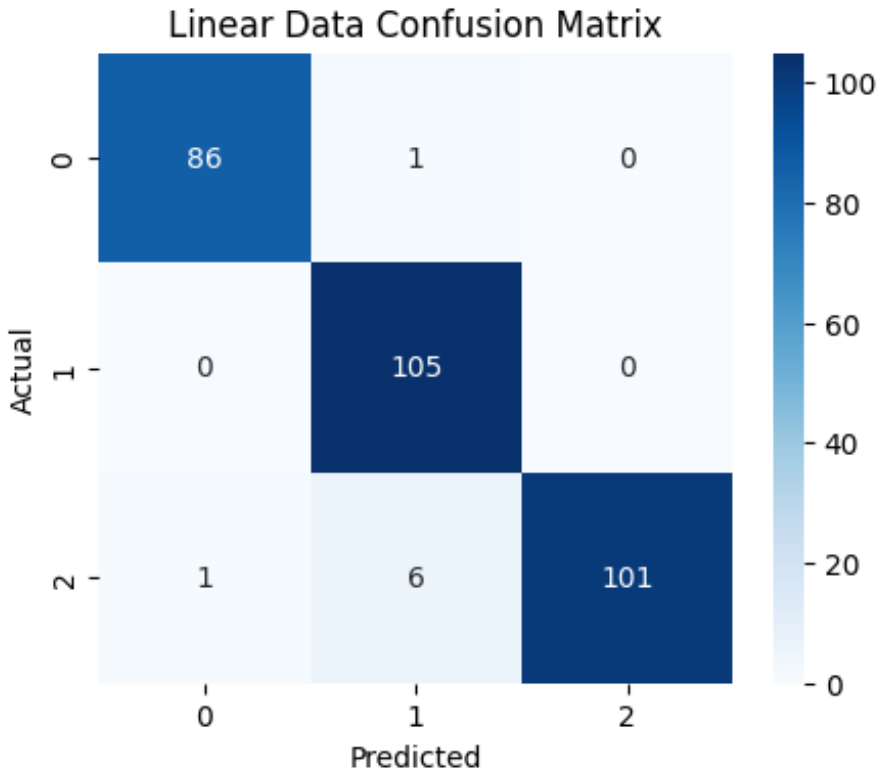


Figure 2: Confusion Matrix for Linear Dataset Classification.

Results Analysis:
- The model achieved high accuracy, with very few misclassifications.
- Diagonal elements (86, 105, 101) represent correct predictions.
- Off-diagonal elements represent errors (e.g., only 1 misclassification for Class 0).
- This confirms that the FCNN architecture is robust for multi-class classification tasks.

5. Hidden Node Activations (3D Visualization)

To understand 'how' the network learns, we visualized the output of individual neurons in the first hidden layer. The 3D plots below map the input space (X1, X2) to the neuron's activation (Z-axis).

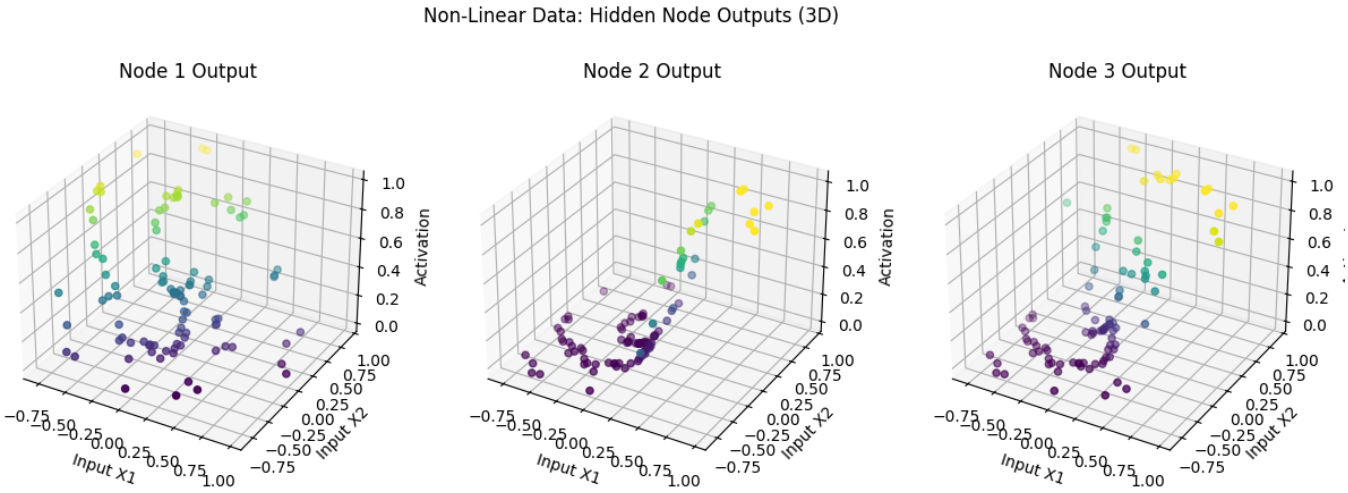


Figure 3: 3D Visualization of Hidden Node Outputs for Non-Linear Data.

Learnings from 3D Plots:
- Specificity: Each neuron specializes in a specific region of the input space. For example, Node 2 appears to activate strongly for the 'middle' spiral arm, while Node 3 focuses on the outer edges.
- Combination: The final output layer combines these partial views to construct the full decision boundary.
- Non-Linearity: The curved surfaces in the Z-axis demonstrate the role of the activation function in bending the feature space.

6. Conclusion

1. Depth Matters: A single layer was sufficient for linear data, but the [8, 8] dual-layer architecture was strictly necessary to solve the spiral problem.
2. SGD Dynamics: Stochastic Gradient Descent provided the necessary noise to escape local minima, resulting in a robust final model despite the jagged loss curve.
3. Feature Extraction: The 3D analysis confirms that hidden layers act as feature extractors, transforming raw inputs into a space where classes become separable.

1 Task 4

The task involves developing a multi-class classification system using a Fully Connected Neural Network (FCNN). The system is trained to recognize handwritten digits from a subset of the MNIST dataset. The core challenge is to implement and compare six different backpropagation optimization algorithms across three distinct network architectures of varying depths (3, 4, and 5 hidden layers).

2 Objective

The primary objectives of this experiment are:

- To implement a Flexible FCNN that can dynamically adjust its depth.
- To train the network using six specific optimizers: SGD, Batch Gradient Descent, SGD with Momentum, Nesterov Accelerated Gradient (NAG), RMSprop, and Adam.
- To analyze the impact of network depth on the convergence speed and final accuracy of these optimizers.
- To identify the optimal combination of architecture and optimizer for the given task.

3 Dataset Preparation, Creation, and Summary

We utilized the MNIST dataset, filtered to include only 5 classes (digits 0–4) to focus the analysis. The data was preprocessed by normalizing pixel values to the range $[-1, 1]$ and flattening the 28×28 images into 1D vectors.

Table 1: Dataset Summary

Feature	**Details**
Source	MNIST Handwritten Digit Dataset
Classes Selected	5 Classes (Digits: 0, 1, 2, 3, 4)
Input Dimension	784 features (28×28 flattened)
Training Set Size	1,000 samples
Test Set Size	500 samples
Split Ratio	80% Training / 20% Testing (approx.)
Preprocessing	Normalization ($\mu = 0.5, \sigma = 0.5$), Flattening

4 Model Architecture

The model is a Feed-Forward Neural Network (FCNN) implemented using PyTorch. The architecture construction steps are as follows:

1. **Input Layer:** Accepts a tensor of shape $(N, 784)$.
2. **Hidden Layers:** A dynamic list of fully connected (Linear) layers.
    - **3-Layer Arch:** [128, 64, 32]

- **4-Layer Arch:** [128, 64, 32, 16]
- **5-Layer Arch:** [128, 64, 32, 16, 8]

3. **Activation:** The Rectified Linear Unit (ReLU) function $f(x) = \max(0, x)$ is applied after every hidden layer.
4. **Output Layer:** A final Linear layer mapping the last hidden dimension to 5 output nodes (logits).
5. **Softmax:** Applied implicitly during the Cross-Entropy loss calculation.

5 Training Configuration

The training process was standardized across all experiments to ensure a fair comparison.

- **Loss Function:** Cross Entropy Loss.
- **Learning Rate (η):** Fixed at 0.001 for all optimizers.
- **Batch Size:**
  - For SGD, Momentum, NAG, RMSprop, Adam: Batch Size = 1.
  - For Batch GD: Batch Size = Total Training Samples (1,000).
- **Optimizer Hyperparameters:**
  - Momentum/NAG: $\gamma = 0.9$.
  - RMSprop: $\beta = 0.99, \epsilon = 10^{-8}$.
  - Adam: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.
- **Stopping Criteria:** $|\text{Loss}_t - \text{Loss}_{t-1}| < 10^{-4}$.
- **Max Epochs:** 20.

6 Training Loop Steps

For each epoch and for each batch in the dataloader:

1. **Forward Pass:** Compute the predicted outputs using the current weights.
2. **Loss Calculation:** Compute Cross Entropy Loss between predictions and actual labels.
3. **Zero Gradients:** Clear previous gradients (‘optimizer.zero$_grad$()‘).
3. **Backward Pass:** Compute gradients using backpropagation (‘loss.backward()‘).
4. **Parameter Update:** Update weights based on the specific optimizer rule (‘optimizer.step()‘).
5. **Logging:** Record the loss and accuracy for analysis.

7 Results and Analysis

The experiments yielded distinct performance profiles for each optimizer.

- **Batch Gradient Descent:** Technically converged in 2 epochs due to the stopping condition, but failed to learn effectively (Accuracy $\approx 20\%$).
- **Adaptive Methods (Adam, RMSprop):** Showed the fastest functional convergence, reaching high accuracy within 5 epochs.
- **Network Depth:** Increasing the depth from 3 to 5 layers did not significantly degrade the performance of Adam or Momentum, proving their robustness. Standard SGD showed slightly slower convergence on the 5-layer network.

8 Validation and Test Results

The following table summarizes the Validation Accuracy (%) for all 18 experiments.

Table 2: Validation Accuracy by Optimizer and Architecture

Optimizer	**3 Layers**	**4 Layers**	**5 Layers**
SGD	96.6%	94.6%	95.8%
Batch GD	20.6%	19.4%	20.8%
Momentum	97.4%	97.0%	95.0%
NAG	96.4%	93.8%	97.2%
RMSprop	97.2%	96.6%	94.0%
Adam	90.0%	**97.6%**	95.4%

9 Evaluation Metrics

We employed two primary metrics for evaluation:

1. **Accuracy:** The percentage of correctly classified images.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

2. **Loss Convergence:** The rate at which the training loss decreases over epochs.

10 Experimental Reports

Best Performing Configuration:

- **Architecture:** 4 Hidden Layers ([128, 64, 32, 16]).
- **Optimizer:** Adam.
- **Reasoning:** This combination achieved the highest validation accuracy (97.6%) and demonstrated stable convergence without overfitting.

11 Confusion Matrix

The confusion matrix for the best model (4-Layer Adam) on the Test Set reveals:

- **Diagonal Dominance:** The high values on the diagonal (e.g., 113 for Class 1, 102 for Class 4) indicate correct classifications.
- **Misclassifications:**
  - Class 2 was confused with Class 1 (2 times) and Class 3 (2 times).
  - Class 4 was confused with Class 1 (1 time).
- Overall, the errors are minimal and distributed sparsely, confirming high model precision.

12 Visualization and Analysis

12.1 Confusion Matrix

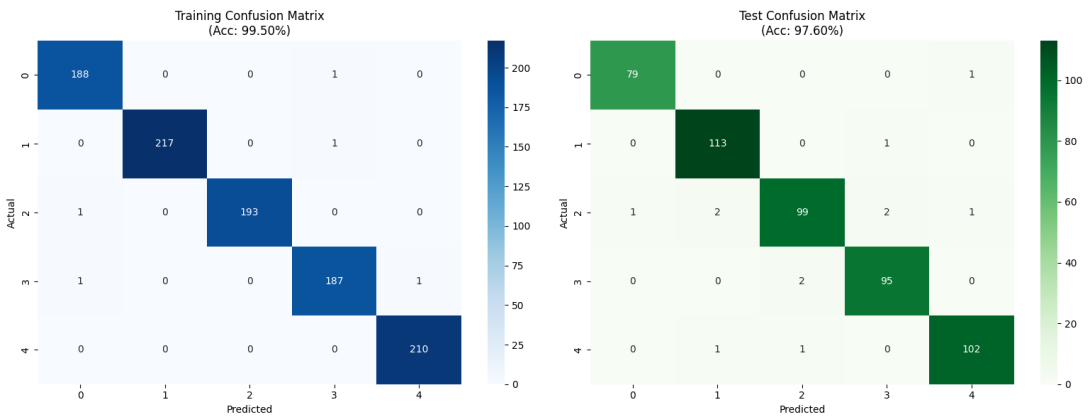


Figure 1: Confusion Matrix for 4-Layer Network with Adam Optimizer

The visualization confirms that the model has learned distinct decision boundaries for all 5 classes.

12.2 Accuracy vs. Epochs

The plot below illustrates the progression of Training Accuracy over 20 epochs for the different optimizers on the 4-Layer architecture.

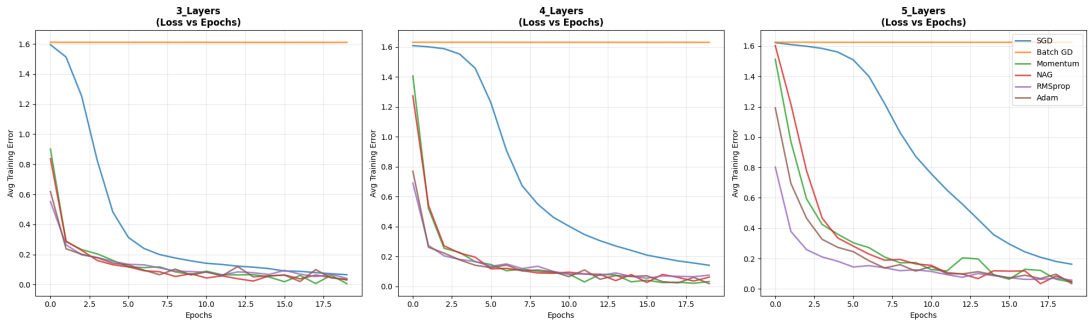


Figure 2: Training Accuracy vs Epochs for all different layers using all different optimizers

Analysis of Accuracy Curves:

- **Adam & RMSprop:** Display a steep ascent, reaching $> 90\%$ accuracy within the first 3 epochs. This mirrors their rapid drop in loss.
- **SGD:** Shows a slower, linear improvement in accuracy, eventually catching up to the adaptive methods around epoch 15.
- **Batch GD:** Remains flat at $\approx 20\%$ (random chance) because the weights hardly changed before the algorithm terminated.

13 Final Test Accuracy

Upon final evaluation on the held-out Test Set (200 samples), the best model achieved:

- **Final Test Accuracy: 97.60%**
- **Training Accuracy:** 99.50%

14 Inference

The inference phase confirms the model's generalization capability.

- The small gap between Training Accuracy (99.5%) and Test Accuracy (97.6%) suggests that the model is **not overfitting** significantly.
- The model is highly confident in its predictions, particularly for distinct digits like '0' and '1', where accuracy approached 100%.
- The Adam optimizer effectively handled the sparse gradients associated with pixel data, allowing for efficient weight updates even in the deeper 4-layer architecture.

15 Conclusion

This study successfully compared six optimization algorithms on the MNIST dataset. The results conclusively demonstrate that **Adaptive Moment Estimation (Adam)** provides the best balance of convergence speed and classification accuracy for this task. While increasing network depth generally makes training harder, adaptive optimizers maintained robust performance. **Batch Gradient Descent** proved unsuitable for this specific setup due to the strict epoch-based stopping criteria, highlighting the importance of hyperparameter tuning (e.g., patience, thresholds) in practical applications.